

Manipulation des requêtes

HTTP

LARAVEL

Introduction



La classe de Laravel **Illuminate\Http\Request** fournit un moyen orienté objet d'interagir avec la requête HTTP actuelle gérée par votre application, ainsi que de récupérer l'entrée, les cookies et les fichiers qui ont été soumis avec la requête.





Interaction avec les requêtes

Interaction avec les requêtes



Pour obtenir une instance de la requête HTTP actuelle via l'injection de dépendances, vous devez indiquer la classe **Illuminate\Http\Request** sur votre méthode de fermeture de route ou de contrôleur. L'instance de requête entrante sera automatiquement injectée par le conteneur de service Laravel :



```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;

class UserController extends Controller
{
    /**
     * Store a new user.
     *
     * @param \Illuminate\Http\Request $request
     * @return \Illuminate\Http\Response
     */
    public function store(Request $request)
    {
        $name = $request->input('name');
        //
    }
}
```

Interaction avec les requêtes



Comme mentionné, vous pouvez également indiquer la classe Illuminate\Http\Request sur une fermeture d'itinéraire. Le conteneur de service injectera automatiquement la requête entrante dans la fermeture lors de son exécution :

```
use Illuminate\Http\Request;

Route::get('/', function (Request $request) {
    //
});
```



Interaction avec les requêtes



Paramètres d'injection de dépendance et de routage

Si votre méthode de contrôleur attend également une entrée d'un paramètre de route, vous devez lister vos paramètres de route après vos autres dépendances. Par exemple, si votre route est définie comme ceci :

```
use App\Http\Controllers\UserController;

Route::put('/user/{id}', [UserController::class, 'update']);
```



Interaction avec les requêtes



Vous pouvez toujours taper le `Illuminate\Http\Request` et accéder à votre paramètre `id route` en définissant votre méthode de contrôleur comme suit :



```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;

class UserController extends Controller
{
    /**
     * Update the specified user.
     *
     * @param \Illuminate\Http\Request $request
     * @param string $id
     * @return \Illuminate\Http\Response
     */
    public function update(Request $request, $id)
    {
        //
    }
}
```

Récupération du chemin de la requête



La méthode **path** renvoie les informations de chemin de la requête. Ainsi, si la requête entrante est ciblée sur **http://example.com/foo/bar**, la méthode **path** retournera **foo/bar**:

```
$uri = $request->path();
```



Inspecter le chemin/la route de la demande



La méthode **is** vous permet de vérifier que le chemin de la demande entrante correspond à un modèle donné. Vous pouvez utiliser le caractère ***** comme caractère générique lors de l'utilisation de cette méthode :

```
if ($request->is('admin/*')) {  
  //  
}
```



Inspecter le chemin/la route de la demande



En utilisant la méthode **routeIs**, vous pouvez déterminer si la requête entrante correspond à une **route nommée** :

```
if ($request->routeIs('admin.*')) {  
    //  
}
```



Récupération de l'URL de la requête



Pour récupérer l'URL complète de la requête entrante, vous pouvez utiliser les méthodes **url** ou **fullUrl**

La méthode `url` renverra l'URL sans la chaîne de requête, tandis que la méthode `fullUrl` inclut la chaîne de requête :

```
$url = $request->url();
```

```
$urlWithQueryString = $request->fullUrl();
```



Récupération de l'URL de la requête



Si vous souhaitez ajouter des données de chaîne de requête à l'URL actuelle, vous pouvez appeler la méthode **fullUrlWithQuery**.

Cette méthode fusionne le tableau donné de variables de chaîne de requête avec la chaîne de requête actuelle :

```
$request->fullUrlWithQuery(['type' => 'phone']);
```



Récupération de l'hôte de requête



Vous pouvez récupérer "l'hôte" de la requête entrante via les méthodes **host**, **httpHost** et **schemeAndHttpHost**:

```
$request->host();  
$request->httpHost();  
$request->schemeAndHttpHost();
```



Récupération de la méthode Request



La méthode **method** renverra le verbe HTTP pour la requête. Vous pouvez utiliser la méthode **isMethod** pour vérifier que le verbe HTTP correspond à une chaîne donnée :

```
$method = $request->method();  
  
if ($request->isMethod('post')) {  
    //  
}
```



En-têtes de demande



Vous pouvez récupérer un en-tête de requête à partir de l'instance **Illuminate\Http\Request** à l'aide de la méthode `header`.

Si l'en-tête n'est pas présent sur la requête, `null` sera renvoyé. Cependant, la méthode `header` accepte un deuxième argument facultatif qui sera retourné si l'en-tête n'est pas présent sur la requête :

```
$value = $request->header('X-Header-Name');
```

```
$value = $request->header('X-Header-Name', 'default');
```



En-têtes de demande



La méthode `hasHeader` peut être utilisée pour déterminer si la requête contient un en-tête donné :

```
if ($request->hasHeader('X-Header-Name')) {  
    //  
}
```



En-têtes de demande



Pour plus de commodité, le procédé **bearerToken** peut être utilisé pour récupérer un jeton de support à partir de l'en-tête **Authorization**. Si aucun en-tête de ce type n'est présent, une chaîne vide sera renvoyée :

```
$token = $request->bearerToken();
```



Demander l'adresse IP



La méthode **ip** peut être utilisée pour récupérer l'adresse IP du client qui a fait la requête à votre application :

```
$ipAddress = $request->ip();
```



Négociation de contenu



Laravel fournit plusieurs méthodes pour inspecter les types de contenu demandés par la requête entrante via l'entête **Accept**. Tout d'abord, la méthode **getAcceptableContentTypes** renverra un tableau contenant tous les types de contenu acceptés par la requête :

```
$contentType = $request->getAcceptableContentTypes();
```



Négociation de contenu



La méthode **accepts** accepte un tableau de types de contenu et renvoie **true** si l'un des types de contenu est accepté par la requête. Dans le cas contraire, **false** sera retourné :

```
if ($request->accepts(['text/html', 'application/json'])) {  
  // ...  
}
```



Négociation de contenu



Vous pouvez utiliser la méthode **prefers** pour déterminer quel type de contenu parmi un tableau donné de types de contenu est le plus préféré par la requête. Si aucun des types de contenu fournis n'est accepté par la requête, **null** sera renvoyé :

```
$preferred = $request->prefers(['text/html', 'application/json']);
```



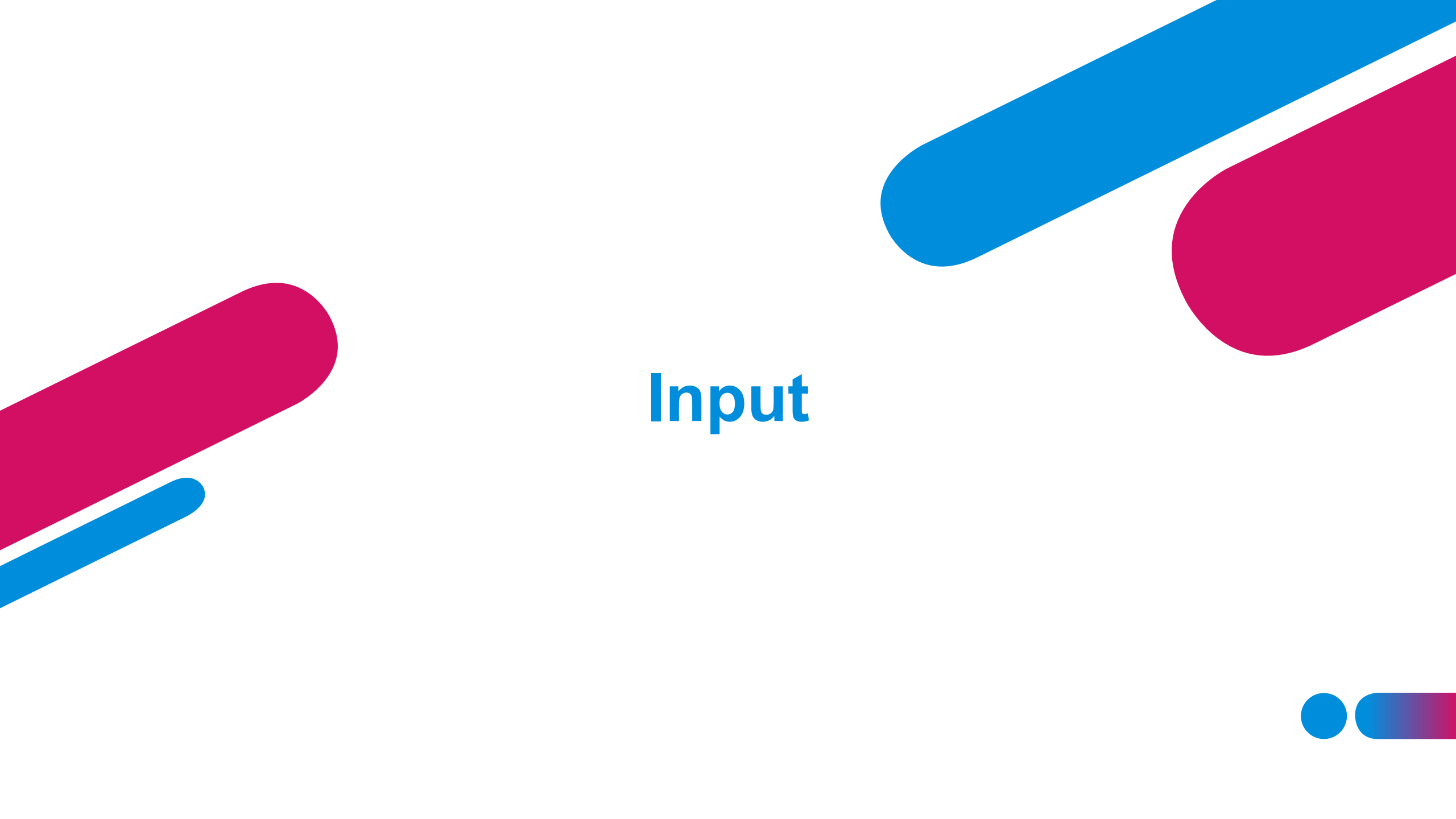
Négociation de contenu



Étant donné que de nombreuses applications ne servent que du HTML ou du JSON, vous pouvez utiliser la méthode **expectsJson** pour déterminer rapidement si la requête entrante attend une réponse JSON :

```
if ($request->expectsJson()) {  
    // ...  
}
```



The background features several thick, rounded, diagonal bars in blue and magenta. On the left, a magenta bar is above a blue bar. On the right, a blue bar is above a magenta bar. In the bottom right corner, there is a small blue circle followed by a horizontal bar with a blue-to-magenta gradient.

Input

Récupération de toutes les données d'entrée



Vous pouvez récupérer toutes les données d'entrée de la demande entrante sous forme d'un **array** en utilisant la méthode **all**. Cette méthode peut être utilisée que la requête entrante provienne d'un formulaire HTML ou soit une requête **XHR** :

```
$input = $request->all();
```



Récupération de toutes les données d'entrée



En utilisant la méthode **collect**, vous pouvez récupérer toutes les données d'entrée de la requête entrante sous la forme d'une collection :

```
$input = $request->collect();
```



Récupération de toutes les données d'entrée



La méthode **collect** vous permet également de récupérer un sous-ensemble de l'entrée de requête entrante sous forme de collection :

```
$request->collect('users')->each(function ($user) {  
    // ...  
});
```



Récupération d'une valeur d'entrée



À l'aide de quelques méthodes simples, vous pouvez accéder à toutes les entrées utilisateur de votre instance **Illuminate\Http\Request** sans vous soucier du verbe HTTP utilisé pour la requête. Quel que soit le verbe HTTP, la méthode `input` peut être utilisée pour récupérer l'entrée utilisateur :

```
$name = $request->input('name');
```



Récupération d'une valeur d'entrée



Vous pouvez passer une valeur par défaut comme second argument de la méthode input. Cette valeur sera renvoyée si la valeur d'entrée demandée n'est pas présente dans la requête :

```
$name = $request->input('name', 'Sally');
```



Récupération d'une valeur d'entrée



Vous pouvez appeler la méthode **input** sans aucun argument afin de récupérer toutes les valeurs d'entrée sous forme de tableau associatif :

```
$input = $request->input();
```



Récupération de l'entrée de la chaîne de requête (Query String)



Alors que la méthode `input` récupère les valeurs de l'intégralité de la charge utile de la requête (y compris la chaîne de requête), la méthode **`query`** ne récupère que les valeurs de la chaîne de requête :

```
$name = $request->query('name');
```



Récupération de l'entrée de la chaîne de requête (Query String)



Si les données de valeur de chaîne de requête demandées ne sont pas présentes, le deuxième argument de cette méthode sera renvoyé :

```
$name = $request->query('name', 'Helen');
```



Récupération de l'entrée de la chaîne de requête (Query String)



Vous pouvez appeler la méthode **query** sans aucun argument afin de récupérer toutes les valeurs de la chaîne de requête sous forme de tableau associatif :

```
$query = $request->query();
```



Récupération des valeurs d'entrée booléennes



Lorsqu'il s'agit d'éléments HTML tels que des **cases à cocher**, votre application peut recevoir des valeurs "véridiques" qui sont en fait des chaînes.

Par exemple, "vrai". Pour plus de commodité, vous pouvez utiliser la méthode **boolean** pour récupérer ces valeurs sous forme de booléens. La méthode **boolean** renvoie true pour 1, "1", vrai, "vrai" et "oui". Toutes les autres valeurs renverront false :

```
$archived = $request->boolean('archived');
```



Récupération des valeurs d'entrée de date

Pour plus de commodité, les valeurs d'entrée contenant des dates/heures peuvent être récupérées en tant qu'une instances à l'aide de la méthode **date**. Si la requête ne contient pas de valeur d'entrée avec le nom donné, **null** sera renvoyé :

```
$birthday = $request->date('birthday');
```

Récupération de l'entrée via les propriétés dynamiques

Vous pouvez également accéder aux entrées utilisateur à l'aide des propriétés dynamiques de l'instance `Illuminate\Http\Request`. Par exemple, si l'un des formulaires de votre application contient un champ `name`, vous pouvez accéder à la valeur du champ comme suit :

```
$name = $request->name;
```

Lors de l'utilisation de propriétés dynamiques, Laravel recherchera d'abord la valeur du paramètre dans la charge utile de la requête. S'il n'est pas présent, Laravel recherchera le champ dans les paramètres de la route correspondante.

Récupération d'une partie des données d'entrée



Si vous avez besoin de récupérer un sous-ensemble des données d'entrée, vous pouvez utiliser les méthodes **only** et **except**. Ces deux méthodes acceptent un seul **array** ou une liste dynamique d'arguments :

```
$input = $request->only(['username', 'password']);
```

```
$input = $request->only('username', 'password');
```

```
$input = $request->except(['credit_card']);
```

```
$input = $request->except('credit_card');
```



Déterminer si l'entrée est présente



Vous pouvez utiliser la méthode **has** pour déterminer si une valeur est présente sur la demande. La méthode **Has** retourne **true** si la valeur est présente sur la requête :

```
if ($request->has('name')) {  
    //  
}
```



Déterminer si l'entrée est présente



Lorsqu'on lui donne un tableau, la méthode has déterminera si toutes les valeurs spécifiées sont présentes :

```
if ($request->has(['name', 'email'])) {  
  //  
}
```



Déterminer si l'entrée est présente



La méthode **whenHas** exécutera la fermeture donnée si une valeur est présente sur la requête :

```
$request->whenHas('name', function ($input) {  
    //  
});
```



Déterminer si l'entrée est présente



Une deuxième fermeture peut être passée à la méthode **whenHas** qui sera exécutée si la valeur spécifiée n'est pas présente sur la requête :

```
$request->whenHas('name', function ($input) {  
    // The "name" value is present...  
}, function () {  
    // The "name" value is not present...  
});
```



Déterminer si l'entrée est présente



La méthode **hasAny** renvoie **true** si l'une des valeurs spécifiées est présente :

```
if ($request->hasAny(['name', 'email'])) {  
  //  
}
```



Déterminer si l'entrée est présente



Si vous souhaitez déterminer si une valeur est présente sur la requête et n'est pas vide, vous pouvez utiliser la méthode **filled** :

```
if ($request->filled('name')) {  
    //  
}
```



Déterminer si l'entrée est présente



La méthode **whenFilled** exécutera la fermeture donnée si une valeur est présente sur la requête et n'est pas vide :

```
$request->whenFilled('name', function ($input) {  
    //  
});
```



Déterminer si l'entrée est présente



Une deuxième fermeture peut être passée à la méthode **whenFilled** qui sera exécutée si la valeur spécifiée n'est pas "remplie":

```
$request->whenFilled('name', function ($input) {  
    // The "name" value is filled...  
}, function () {  
    // The "name" value is not filled...  
});
```



Déterminer si l'entrée est présente



Pour déterminer si une clé donnée est absente de la requête, vous pouvez utiliser la méthode **missing** :

```
if ($request->missing('name')) {  
    //  
}
```



Ancienne entrée



Laravel vous permet de **conserver les entrées d'une requête** lors de la prochaine requête. Cette fonctionnalité est particulièrement utile pour remplir à nouveau les formulaires après avoir détecté des erreurs de validation. Cependant, si vous utilisez les fonctionnalités de validation incluses de Laravel, il est possible que vous n'ayez pas besoin d'utiliser manuellement ces méthodes de flashage d'entrée de session directement, car certaines des fonctionnalités de validation intégrées de Laravel les appelleront automatiquement.



Entrée clignotante à la session



La méthode flash sur la classe **Illuminate\Http\Request** fera clignoter l'entrée actuelle de la session afin qu'elle soit disponible lors de la prochaine requête de l'utilisateur à l'application :

```
$request->flash();
```



Entrée clignotante puis redirection

Étant donné que vous souhaitez souvent flasher l'entrée dans la session, puis rediriger vers la page précédente, vous pouvez facilement enchaîner l'entrée clignotante sur une redirection en utilisant la méthode **withInput** :

```
return redirect('form')->withInput();
```

```
return redirect()->route('user.create')->withInput();
```

```
return redirect('form')->withInput(  
$request->except('password')  
);
```

Récupération de l'ancienne entrée



Pour récupérer l'entrée flashée de la requête précédente, appelez la méthode **old** sur une instance de **Illuminate\Http\Request**.

La méthode **old** extraira les données d'entrée précédemment flashées de la session :

```
$username = $request->old('username');
```



Récupération de l'ancienne entrée

Laravel fournit également une fonction Helper globale **old**. Si vous affichez d'anciennes entrées dans un modèle Blade, il est plus pratique d'utiliser Le Helper **old** pour remplir à nouveau le formulaire. Si aucune ancienne entrée n'existe pour le champ donné, **null** sera renvoyé :

```
<input type="text" name="username" value="{{ old('username') }}">
```

Récupération des cookies à partir des demandes

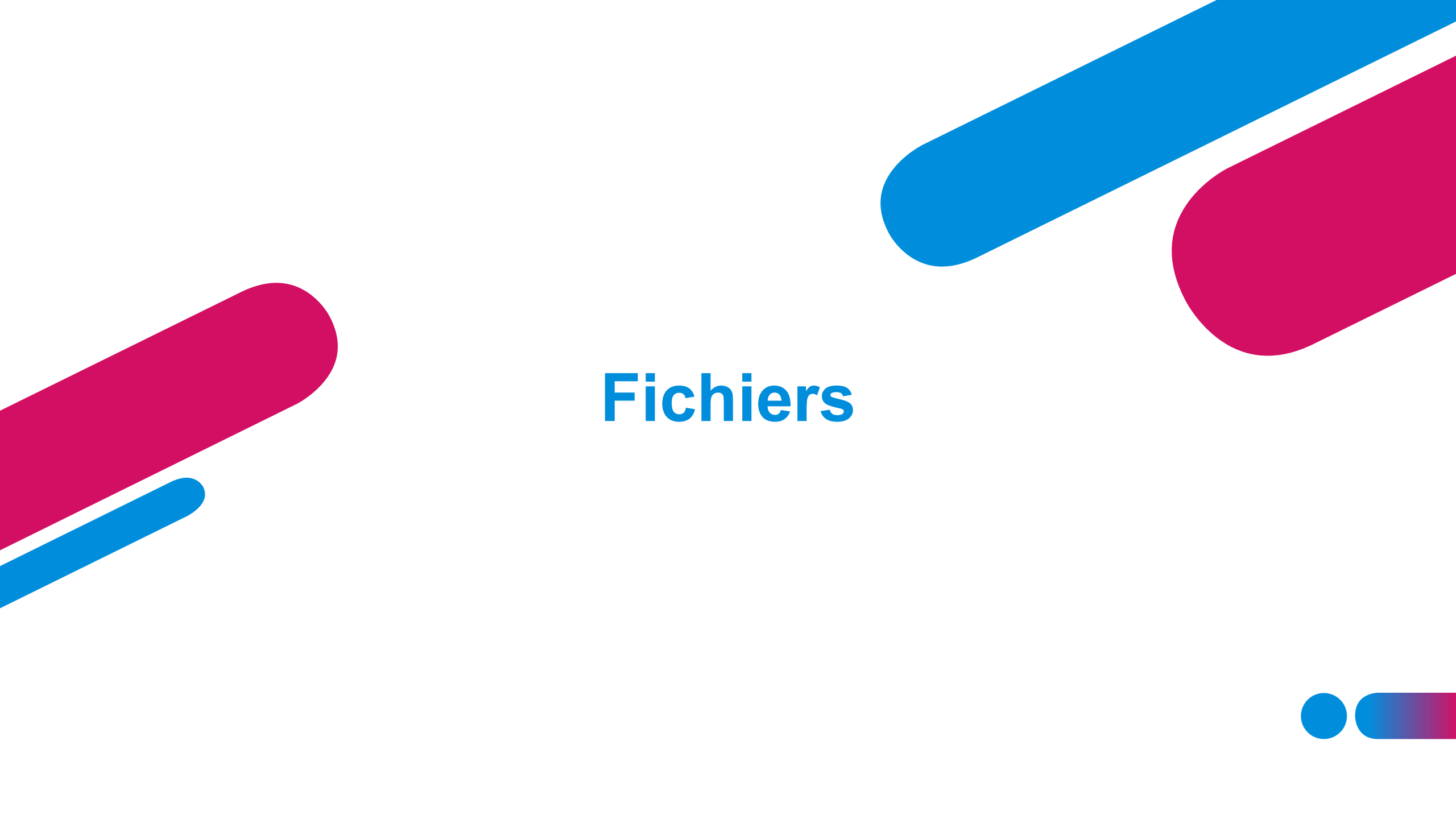


Tous les cookies créés par le framework Laravel sont cryptés et signés avec un code d'authentification, ce qui signifie qu'ils seront considérés comme invalides s'ils ont été modifiés par le client.

Pour récupérer une valeur de cookie à partir de la requête, utilisez la méthode **cookie** sur une instance **Illuminate\Http\Request** :

```
$value = $request->cookie('name');
```





Fichiers



Récupération des fichiers téléchargés



Vous pouvez récupérer des fichiers téléchargés à partir d'une instance **Illuminate\Http\Request** à l'aide de la méthode **file** ou à l'aide de propriétés dynamiques. La méthode **file** renvoie une instance de la classe **Illuminate\Http\UploadedFile**, qui étend la classe PHP **SplFileInfo** et fournit une variété de méthodes pour interagir avec le fichier :

```
$file = $request->file('photo');
```

```
$file = $request->photo;
```



Récupération des fichiers téléchargés



Vous pouvez déterminer si un fichier est présent sur la requête en utilisant la méthode **hasFile** :

```
if ($request->hasFile('photo')) {  
  //  
}
```



Validation des téléchargements réussis



En plus de vérifier si le fichier est présent, vous pouvez vérifier qu'il n'y a eu aucun problème lors du téléchargement du fichier via la méthode **isValid** :

```
if ($request->file('photo')->isValid()) {  
    //  
}
```



Chemins de fichiers et extensions



La classe **UploadedFile** contient également des méthodes pour accéder au chemin complet du fichier et à son extension.

La méthode **extension** tentera de deviner l'extension du fichier en fonction de son contenu. Cette extension peut être différente de l'extension fournie par le client :

```
$path = $request->photo->path();
```

```
$extension = $request->photo->extension();
```



Stockage des fichiers téléchargés



Pour stocker un fichier téléchargé, vous utiliserez généralement l'un de vos systèmes de fichiers configurés . La classe **UploadedFile** a une méthode `store` qui déplacera un fichier téléchargé vers l'un de vos disques, qui peut être un emplacement sur votre système de fichiers local ou un emplacement de stockage cloud.



Stockage des fichiers téléchargés

La méthode **store** accepte le chemin où le fichier doit être stocké par rapport au répertoire racine configuré du système de fichiers. Ce chemin ne doit pas contenir de nom de fichier, car un identifiant unique sera automatiquement généré pour servir de nom de fichier.

La méthode **store** accepte également un deuxième argument facultatif pour le nom du disque qui doit être utilisé pour stocker le fichier. La méthode renverra le chemin du fichier par rapport à la racine du disque :

```
$path = $request->photo->store('images');  
$path = $request->photo->store('images', 's3');
```

Stockage des fichiers téléchargés



Si vous ne souhaitez pas qu'un nom de fichier soit généré automatiquement, vous pouvez utiliser la méthode **storeAs**, qui accepte le chemin, le nom de fichier et le nom du disque comme arguments :

```
$path = $request->photo->storeAs('images', 'filename.jpg');
```

```
$path = $request->photo->storeAs('images', 'filename.jpg', 's3');
```

